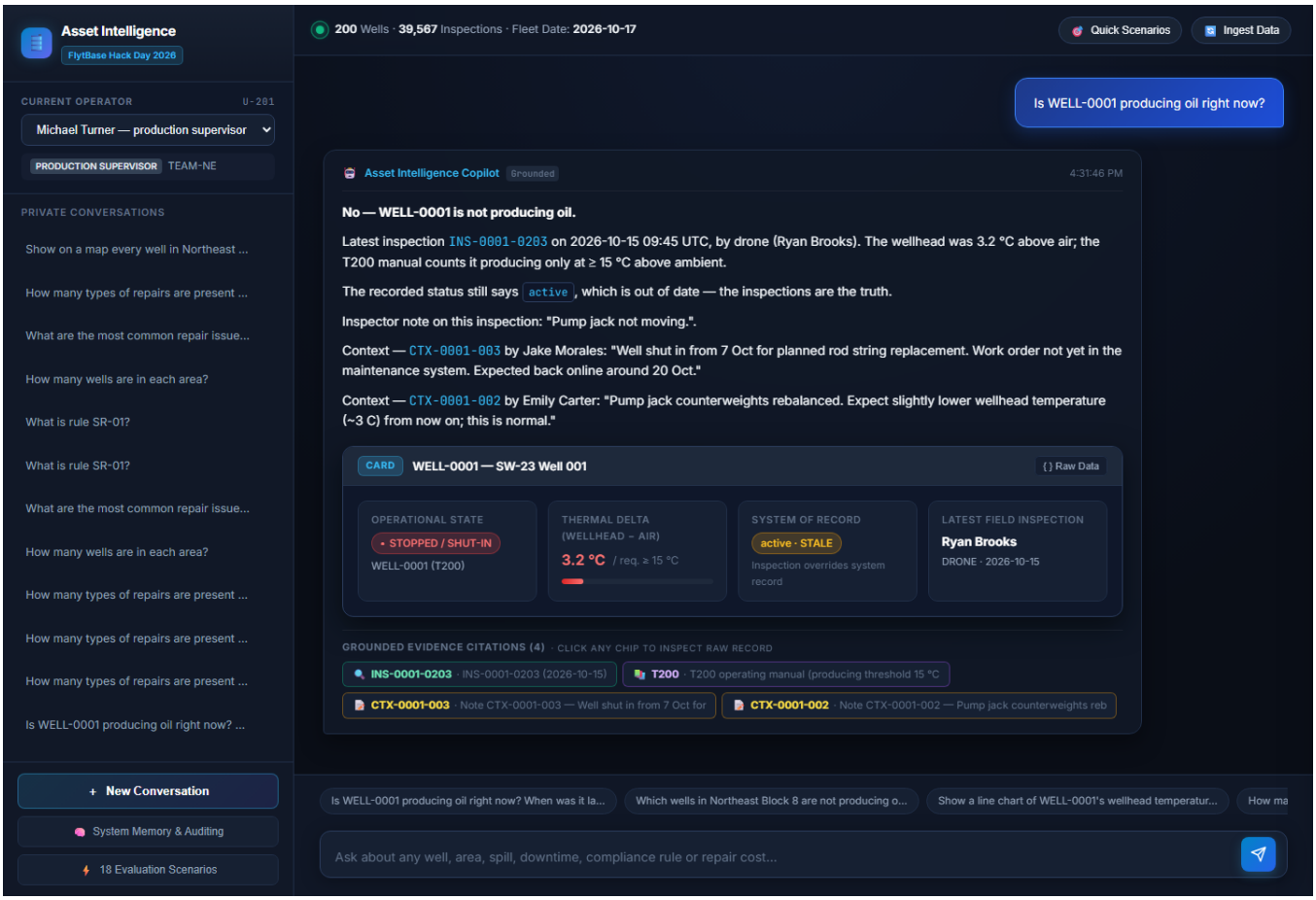
Asset Intelligence Copilot

Detailed project walkthrough | FlytBase Local Challenge | Hacktoberfest 2026

Project	**Document role**
A local operations copilot that connects inspections, maintenance, spill reports, manuals, SOPs, and operator notes.	Supplemental implementation guide. Keep the event-required two-page design/results report as the primary write-up.

Real frontend capture: an operator asks whether WELL-0001 is producing. The app compares the latest thermal inspection with the T200 manual, flags stale recorded status, and displays clickable evidence.



Reading this answer: a 3.2 C delta is below the 15 C T200 producing threshold. The card marks the well stopped/shut-in and the recorded “active” value stale. The answer cites inspection INS-0001-0203, manual T200, and operator notes CTX-0001-003 and CTX-0001-002.

1. System architecture

The browser UI calls a FastAPI service. A deterministic router selects Python domain tools over a unified SQLite database. The response returns as one envelope containing readable text, optional visual data, and evidence references.



Design choice: reproducible domain arithmetic stays in Python; routing, memory, and document retrieval connect the mixed data sources; visual selection happens per answer rather than through fixed dashboard pages.

Key modules: main.py (API), db.py (schema/ingestion), nlu.py (prompt parsing), agent.py (orchestration), domain.py (rules), tools.py (queries/calculations), memory.py (shared/private state), and frontend/index.html (single-page UI).

2. Data ingestion and storage

The ingestor reads the supplied fleet directory structure and normalizes it into SQLite. Natural record IDs are retained for citations. Multiple roots can be merged; INSERT OR REPLACE lets the loader be rerun without duplicating canonical records.

Source	Fields and content
fleet/areas.json, users.json	Areas, teams, locations, roles, and operator identities.
fleet/well-types/*.md	Well-type manuals and producing thresholds.
fleet/sops/*.md	Spill response and regulatory/field-work procedures.
assets/*/metadata.json	Well type, area, team, coordinates, equipment, recorded status.
inspections.jsonl	Weekly wellhead/ambient temperatures, inspector, spill detections, notes.
maintenance.jsonl, reports/*.md	Repair issue/action/costs and parsed free-text containment/RCA facts.
regulatory-events.jsonl, context.jsonl	Notices/fines/visits plus human notes, corrections, and planned work.

Database model

Purpose	Tables
Field reference	wells, areas, teams, users
Operational records	inspections, maintenance, reg_events, reports, docs, context_notes
Conversation/memory	conversations, messages, entity_mentions, corrections, shared_notes, claims, user_rules

SQLite opens in WAL mode with indexes on asset/date query paths. The live frontend displayed 200 wells and 39,567 inspections. The default database is backend/copilot.db; COPILOT_DB can select another file.

3. From prompt to answer

Step	What happens
Receive	POST /api/chat resolves the active operator and passes the question to agent.answer.
Parse	nlu.route identifies intent, well IDs, area, equipment, time window, limits, and requested visual.
Scope memory	Private entity references resolve only for that user; applicable shared notes/corrections are loaded.
Dispatch	agent._dispatch picks a domain operation such as well_status, downtime_ranking, spill_map, or heatmap.
Compute	tools.py reads SQLite and applies domain.py rules; output contains text/data plus evidence IDs.
Render	The frontend selects a visual renderer and draws answer text, visualization, and evidence chips.

AI provider box for the event form

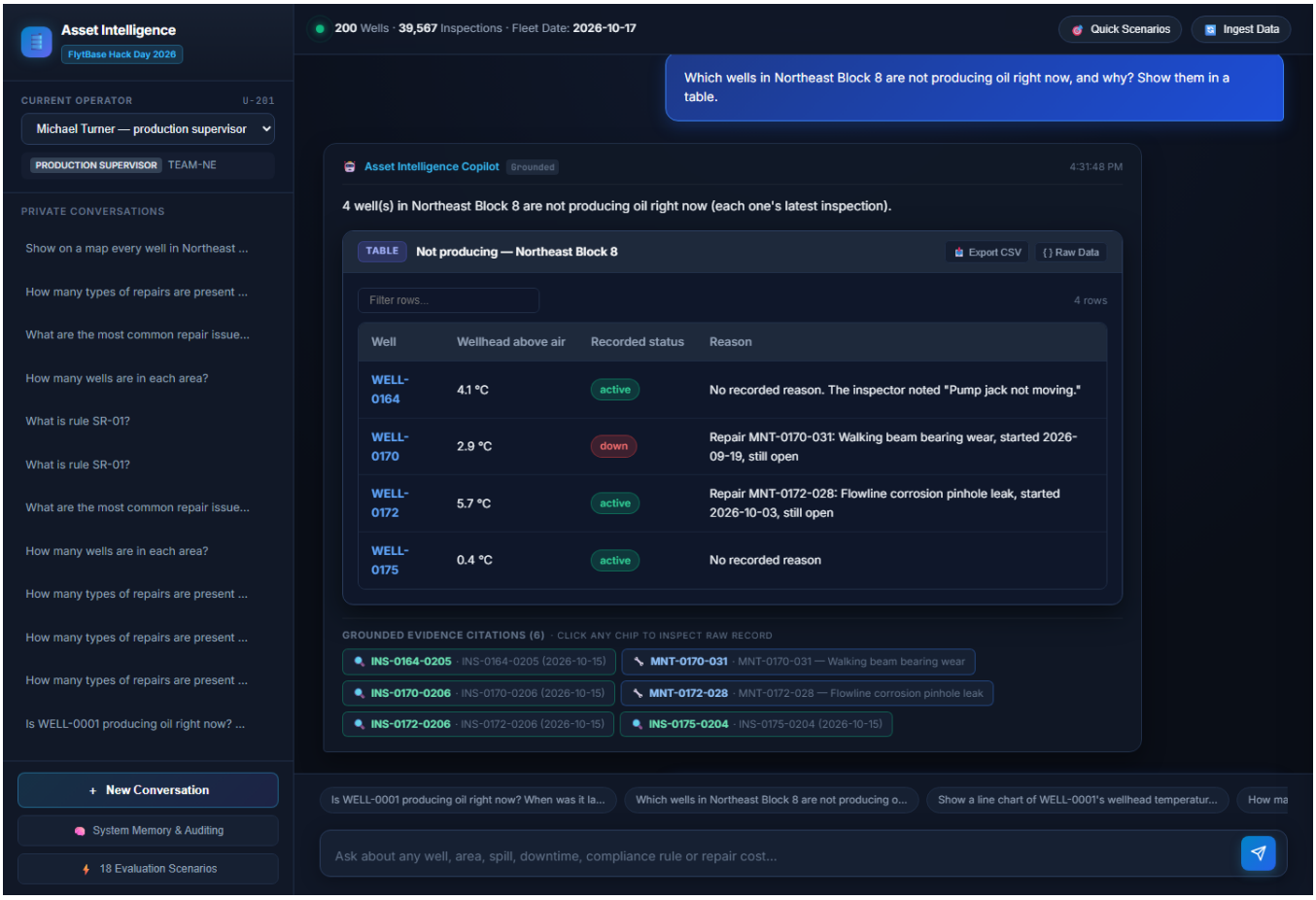
For the running local demo inspected for this guide, select none. Its Python listener process had no GEMINI_API_KEY, OPENAI_API_KEY, or LLM_API_KEY configured. The demonstrated path uses the built-in deterministic router and Python tools without calling an external model provider.

If you configure this for the final demo	Portal selection
GEMINI_API_KEY is set and Gemini is called	Google Gemini
OPENAI_API_KEY is set and OpenAI is called	OpenAI
LLM_API_KEY + custom base URL	Select the provider actually behind it, or Other.
No provider key in the runtime	Leave all app-provider boxes unchecked.

The optional OpenAI-compatible path can generate SQL for otherwise-unhandled queries and optionally rephrase prose. Its provider/model/cost depends on runtime settings. The no-key capture in this guide does not claim model usage or API cost. Coding-assistant selection is a separate form field.

4. Frontend example: area table

The area query returns the wells not producing at their latest inspection, with thermal delta, stale recorded status, reason, source evidence, row filtering, and CSV export. Repairs remain linked to the relevant well rows.

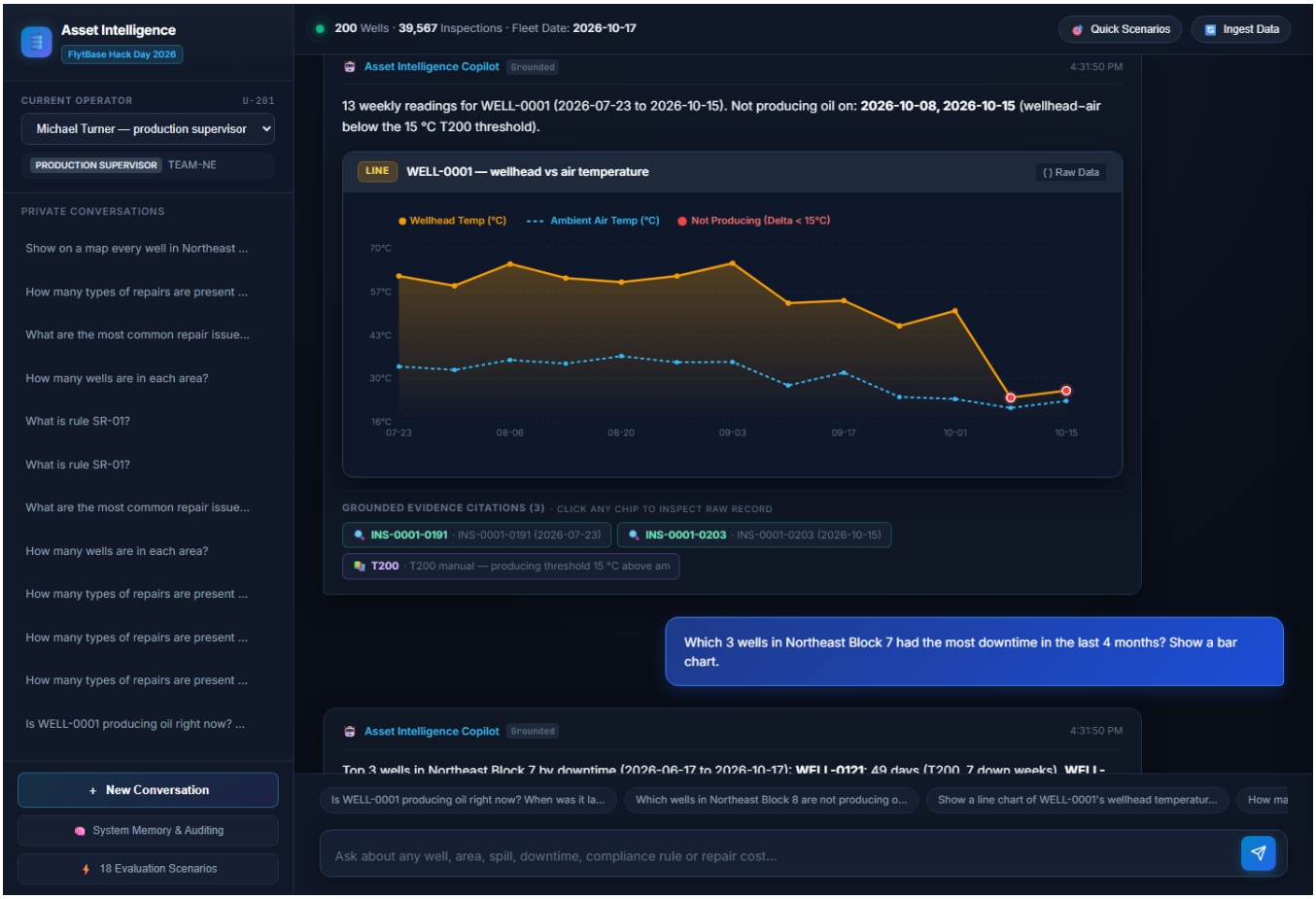


What the answer means: four wells in Northeast Block 8 are below their well-type threshold. Two still show “active” in the source metadata; two have open repairs linked in the reason column. This list is calculated from current records, not hard-coded into the visual.

Code path: tools.list_not_producing creates rows/evidence; renderTable displays the answer envelope.

5. Frontend example: temperature trend

The chart query reads the most recent 13 weekly inspections, plots wellhead and ambient temperature as separate series, and highlights dates below the producing threshold.

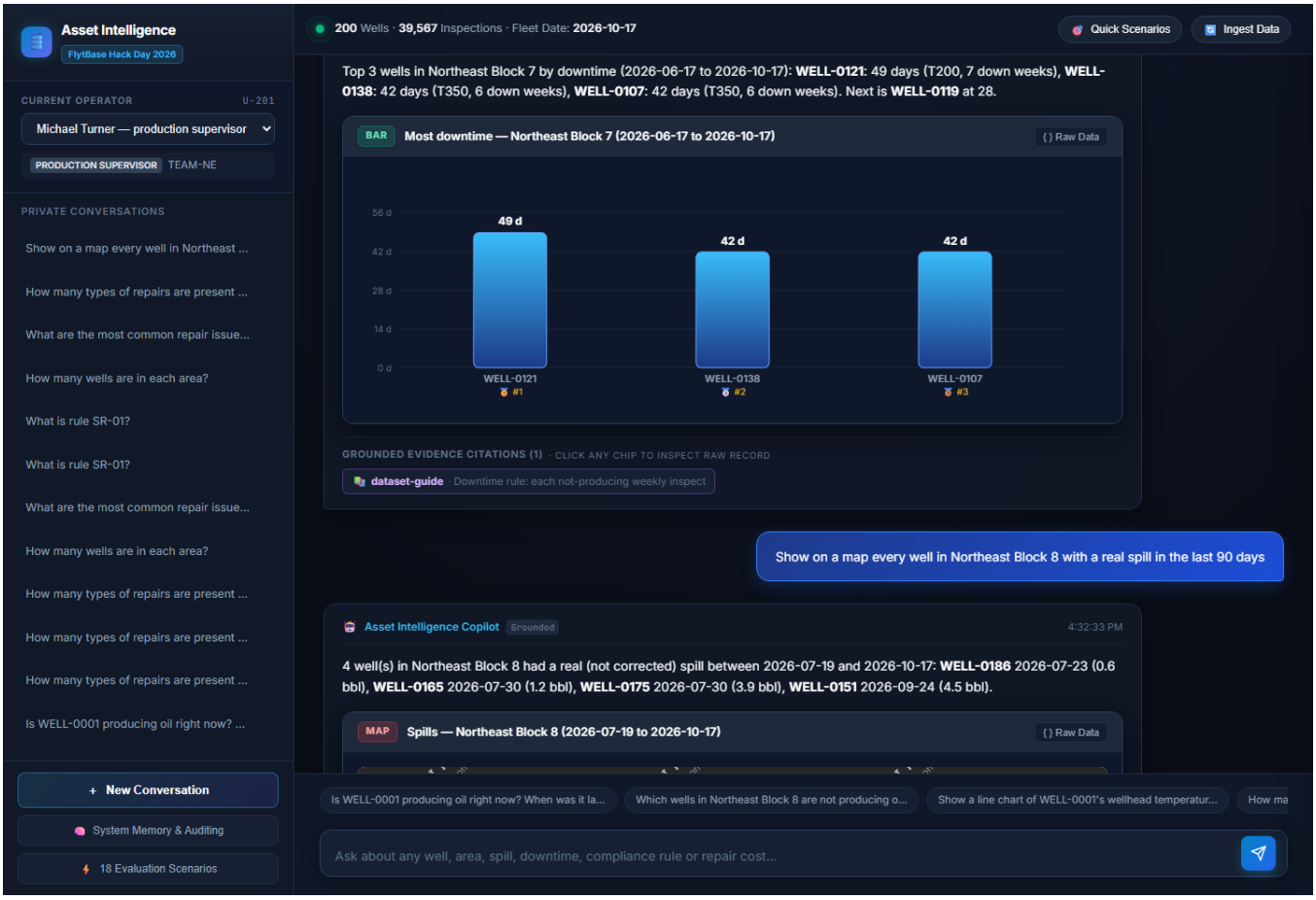


Visible result: the final two readings, 8 Oct and 15 Oct 2026, are marked not producing because the delta is under the T200 threshold. The response cites readings and the manual.

Code path: tools.temp_series builds the time series and non-producing date list; renderLineChart draws it from the visual payload.

6. Frontend example: downtime ranking

The downtime bar chart ranks wells by weekly inspections showing non-production. Each such inspection contributes seven downtime days. A missed inspection contributes none; maintenance duration does not replace this challenge rule.

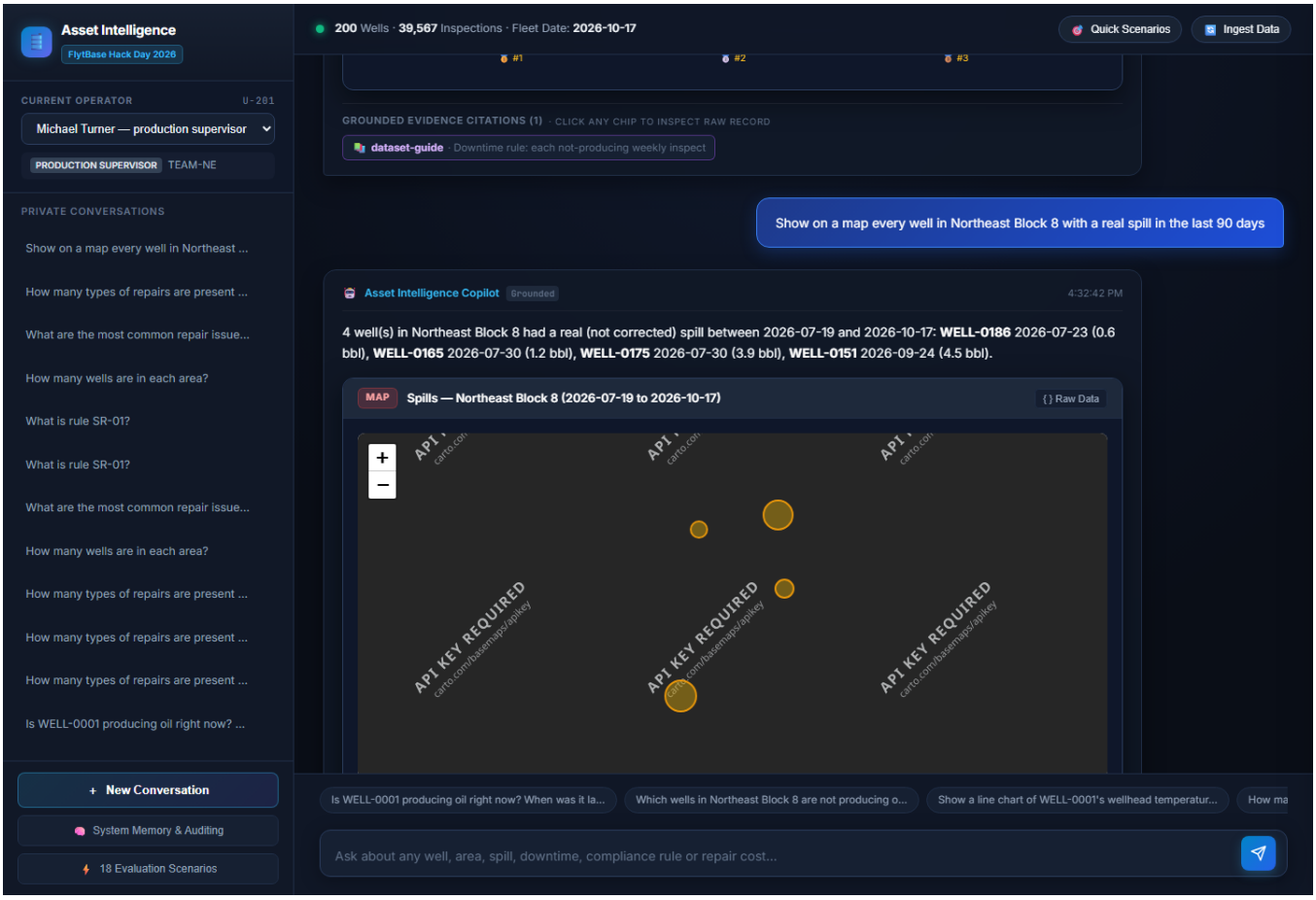


Visible result: WELL-0121 has 49 days, WELL-0138 has 42, and WELL-0107 has 42 for the selected four-month window. The next-ranked well gives context.

Code path: tools.downtime_ranking aggregates the per-well downtime calculation; renderBarChart adds ranking labels.

7. Frontend example: spill map

The map tool filters valid detections by area/date, returns coordinates and volumes, then draws circle markers sized by spill volume with a detail popup.

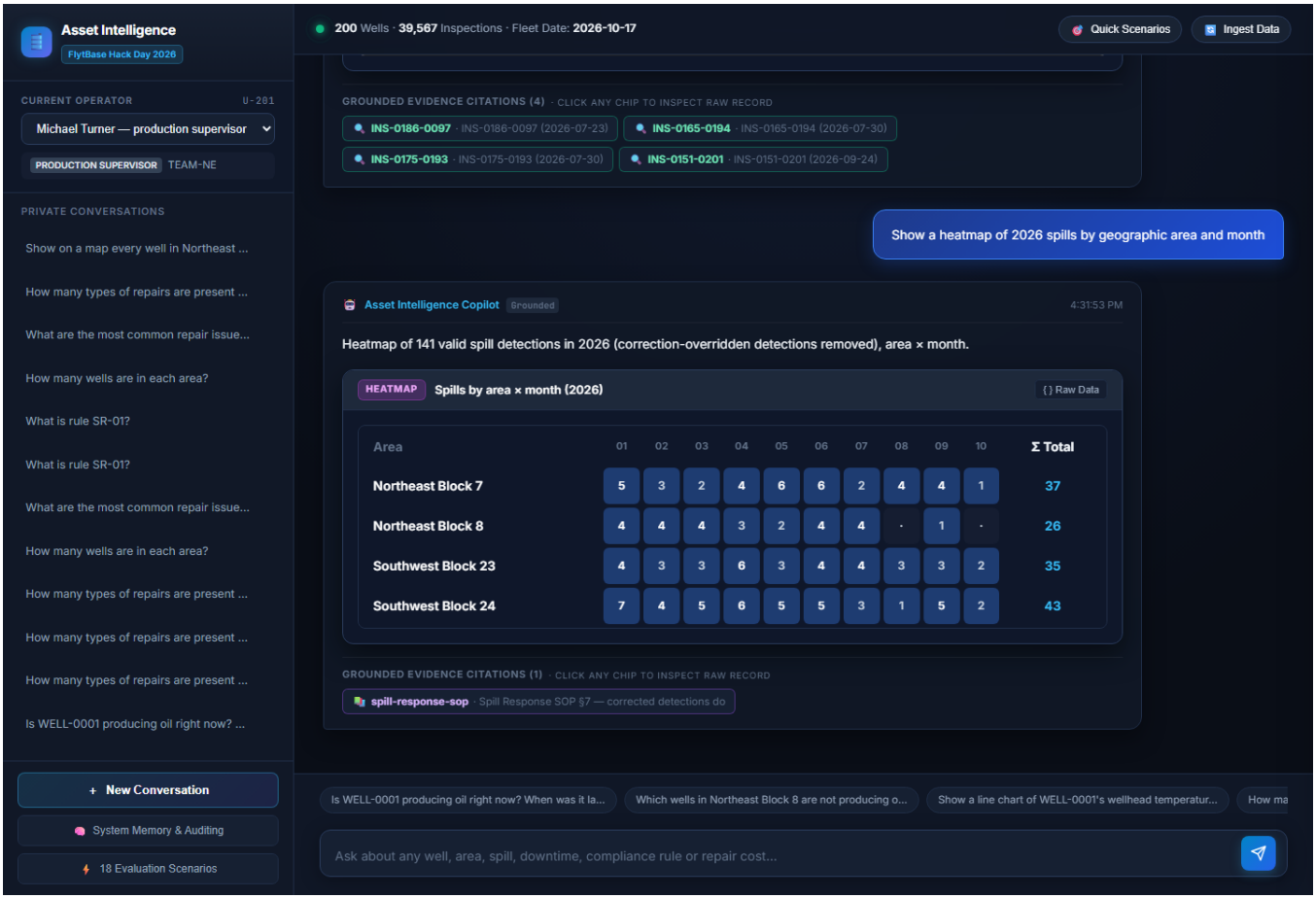


Capture limitation: the pin layer appears, but the basemap provider returned an “API KEY REQUIRED” placeholder. The map data is present; configure or replace the background tile service before depending on a public map demo. An SVG fallback exists if Leaflet itself cannot load.

Code path: tools.spill_map -> renderMap / initLeafletMap.

8. Frontend example: spill heatmap

For fleet-wide comparison, the heatmap groups valid detections by area and month. Corrections that invalidate an original detection are excluded, and each area row shows a total.



Visible result: 141 valid 2026 detections: Northeast Block 7 (37), Northeast Block 8 (26), Southwest Block 23 (35), Southwest Block 24 (43).

Code path: tools.spill_heatmap builds the matrix; renderHeatmap presents the cells, totals, and intensity scale.

9. Memory, corrections, and conflict handling

Shared field context is intentionally distinct from an operator's private conversation references. Corrections preserve scope and provenance, so the app can use a corrected value later without erasing the original.

Mechanism	Storage/scope	Behavior
Private memory	entity_mentions by user/conversation	"The wells we discussed" resolves only from that user's own history.
Shared knowledge	shared_notes by well	A planned shutdown note is available to other users asking about that well.
Raw-data correction	corrections by inspection/well	Keeps original/new value, author, reason, and later-answer effect.
Scoped rule	user_rules by well type or well	A T350 rule does not spill over to T200.
Operator claim	claims with conflict state	Contradictory statements are shown as unconfirmed pending review.

Evidence and honesty

Answer envelopes carry evidence descriptors for inspection, repair, document, note, or event sources. The frontend renders clickable evidence chips; /api/evidence/{item_id} returns the raw source record. memory.check_claim_against_data compares claims against available data and surfaces a disagreement rather than silently treating the claim as ground truth.

Example: a claim that WELL-0175 is "running fine" conflicts with a 0.4 C thermal delta. The interface can mark the claim unconfirmed and cite the inspection that triggered the disagreement.

10. Code map and local run

File	Role
backend/app/main.py	FastAPI endpoints and static frontend.
backend/app/db.py	SQLite schema, WAL, ingestion, document/report parsing.
backend/app/nlu.py	Well/area/equipment/date/visual extraction and intent routing.
backend/app/agent.py	Conversation lifecycle, dispatch, corrections, optional model path.
backend/app/domain.py	Threshold lookup, effective corrected readings, valid spills, date windows.
backend/app/tools.py	Status, trends, spills, downtime, compliance, repair costs, summaries.
backend/app/memory.py	Shared notes, private references, rules, corrections, claim conflicts.
frontend/index.html	Single-page chat, six visual renderers, drawers, evidence and scenario UI.
scripts/run_eval.py	Scenario assertions.

Run locally

Install Python 3.10+ and the packages in backend/requirements.txt. From backend, run `python -m uvicorn app.main:app --port 8000`; open `http://localhost:8000`. The app initializes its schema and ingests the configured data roots if the wells table is empty.

Useful endpoints

POST /api/chat; GET /api/users; GET /api/areas; GET /api/conversations; GET /api/conversations/{id}/messages; GET /api/evidence/{item_id}; GET /api/memory; POST /api/ingest.

11. Evaluation record and next steps

The repository records 63/63 assertion checks across 18 supplied scenarios. Coverage includes stale well status, area filtering, trends, downtime, spills/compliance, memory sharing/isolation, corrections, and contradictory claims. The evaluator is scripts/run_eval.py. This guide uses saved frontend examples and did not rerun the evaluator.

Strengths evidenced in the project	Limits/next work
Deterministic status, downtime, spill, and compliance calculations.	Validate route coverage and ingestion against unseen final data.
Source IDs carried into answer citations.	Harden the optional SQL fallback with a read-only DB connection, SQL authorizer/parser, and bounded results.
Shared notes and private memory have separate scopes.	Configure/replace the basemap tile service for public demo use.
Six inline visuals match common question types.	The project analyzes records and text; it does not recognize drone imagery.

Submission pairing

The event asks for a design/results write-up capped at two pages. Submit docs/System_Design_and_Results_Writeup.pdf as the primary report. This longer PDF is a supplemental project walkthrough with live frontend captures and implementation details. If the portal has an “Other links” field, label this as “Detailed project working guide.”

AI app-provider selection for the current local demo: **none**. Only choose a provider if the environment used for the actual demonstration is configured to call it.

Repository: github.com/siddhant-pardeshi/asset-intelligence-copilot | License: MIT | Local demo: http://localhost:8000